

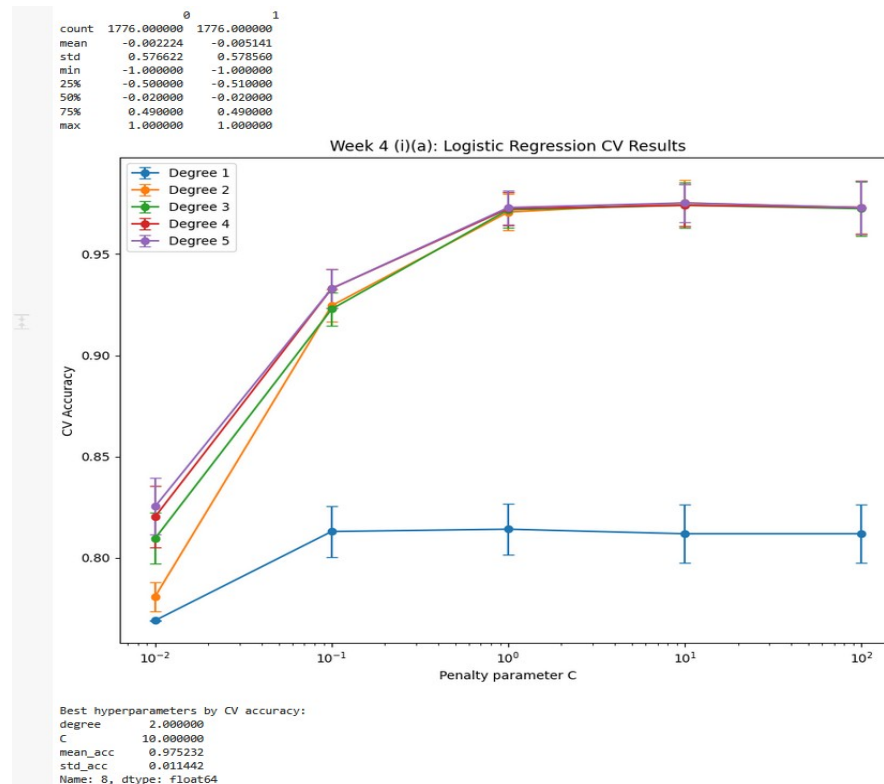
Machine Learning Week 4 Assignment

By Gaurav Naudiyal (Student ID: 25339213)

Msc Computer Science – Augmented & Virtual Reality

my dataset id 1 is : # id:17--34-17-0 and dataset id2 is : # id:17-17-17-0

(i)(a) For this task I used the polynomial feature expansion and logistic regression methods to classify the dataset. To select the most optimal model, I applied 5-fold cross-validation method over a grid of polynomial degree from 1 to 5 and penalty values C from 0.001 to 100, measuring mean classification accuracy for each combination. I have also included error bars to visualize variability across the fields and the central tendency aswell.

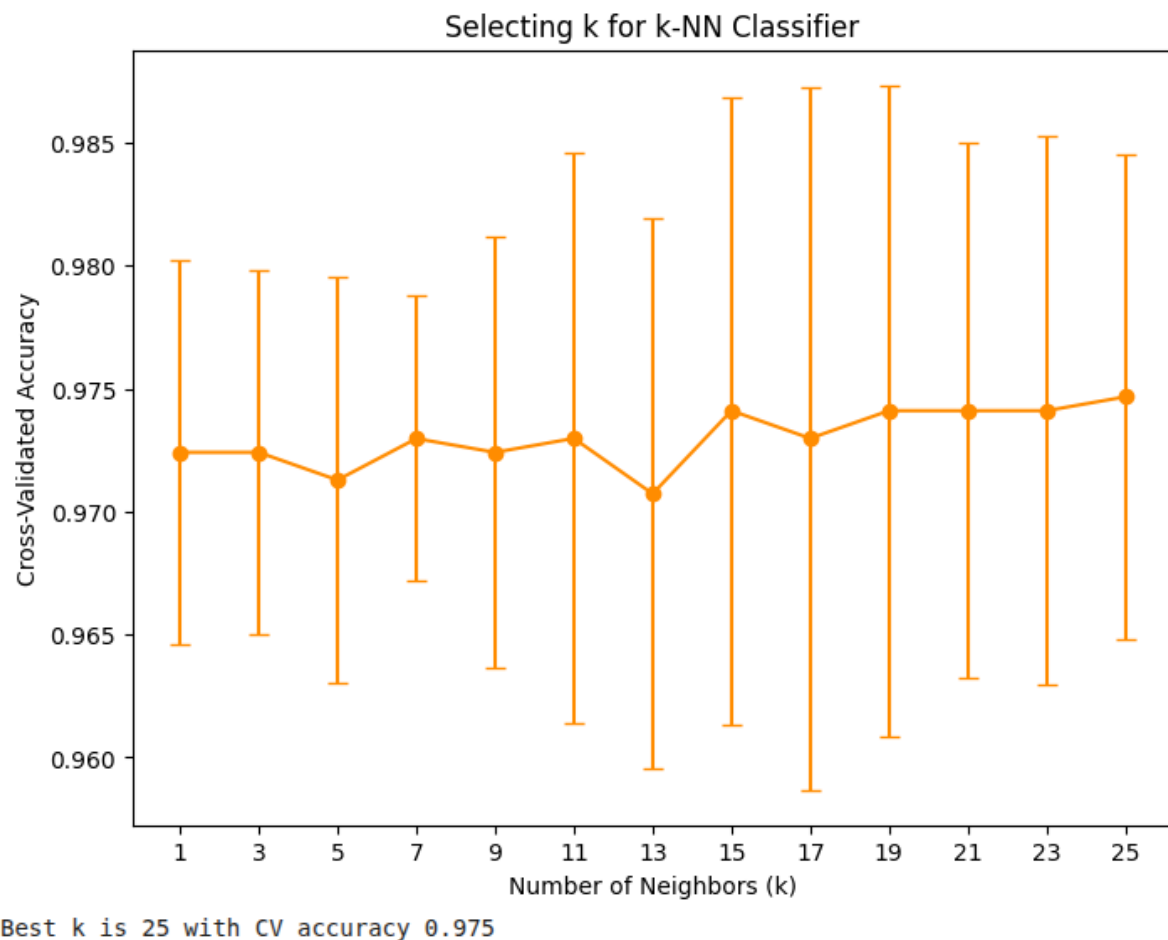


This resulted in a plot showing steep movement in accuracy from 1 to 2, especially as C increases. Degrees 2,3,4 and 5 all achieve higher accuracy for higher values of C.

Degree 1 consistently underperforms the higher degree models which confirms the value of nonlinear features for the dataset.

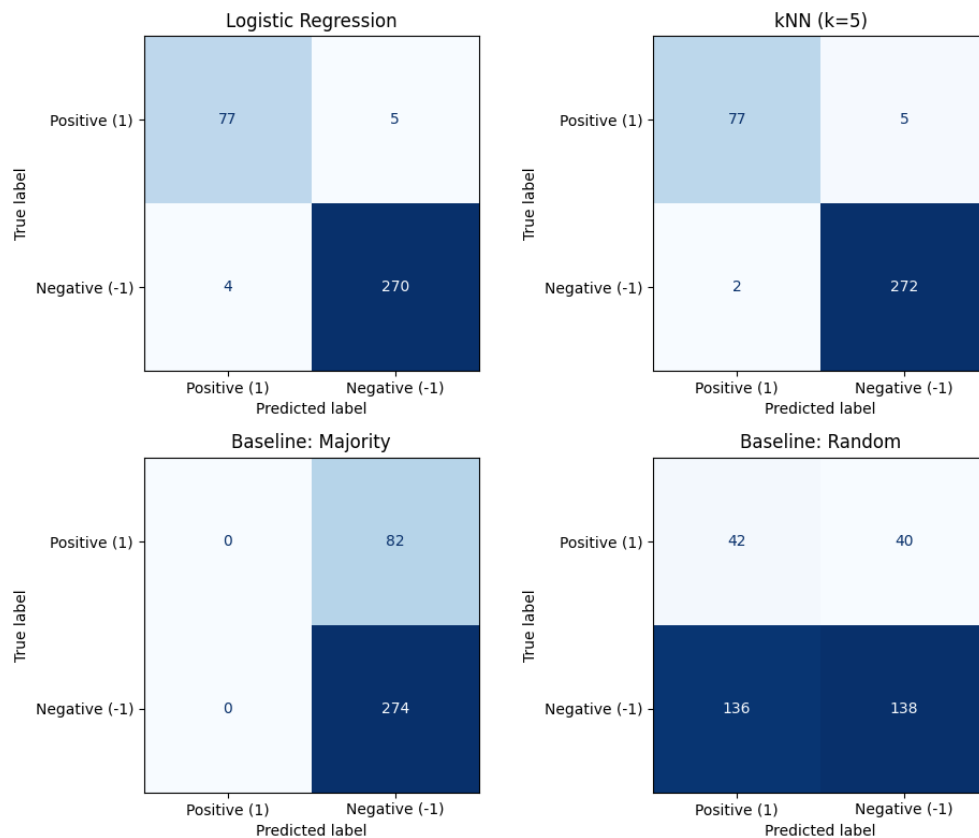
The best mean CV value is at $C=10$ (degree 2). The standard deviation of accuracy is very low here indicating stable performance.

(b) For the knn classifier, I varied the number of neighbors k from 1 to 25 using 5 fold cross validation. I Found that the best parameter of k was selected based on the highest mean cross validation value. This approach shows that knn can capture non linear decision boundaries without any feature augmentation.



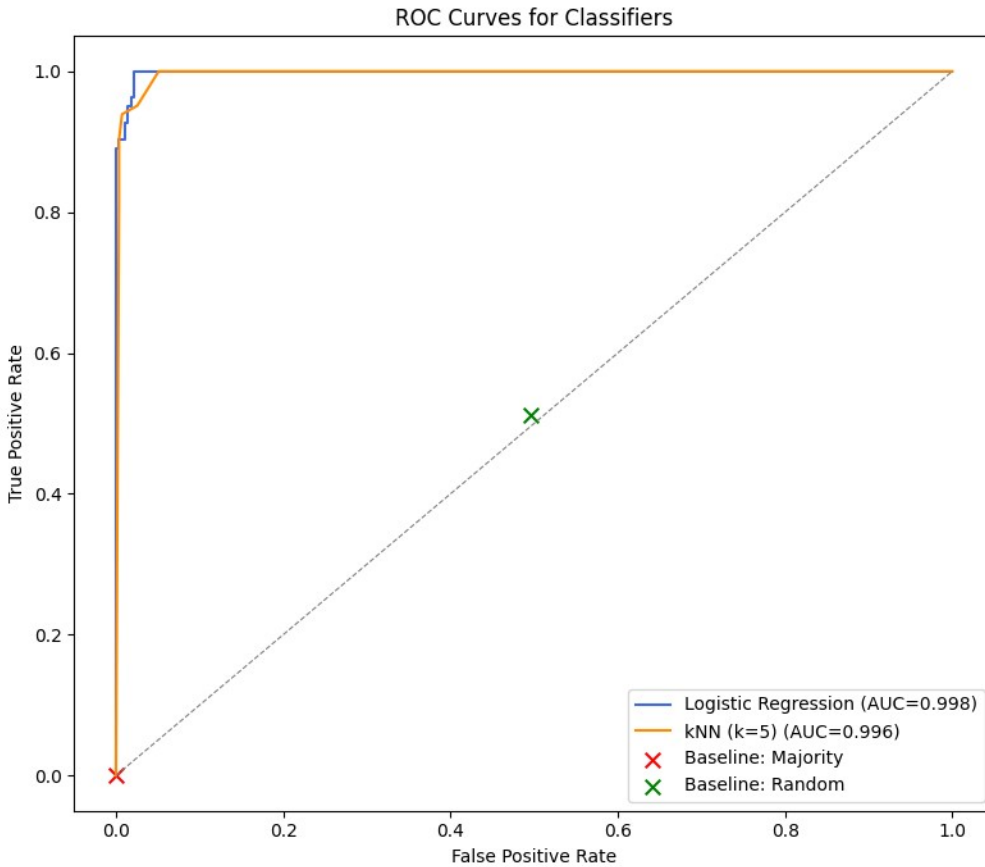
The best value of k was found to be 25 with cv accuracy of 0.975, balancing bias and variance in the model.

(c)



After doing confusion matrices by training logistic regression, knn and two baseline models. I Found out that both logistic regression and kNN generalize very well and are robust against class imbalance. They only have a few errors. The baselines highlight why a real predictive model is necessary and random guessing is not useful for practically identifying minority class. Both precision and recall in random baselines are close to 50%, which demonstrates the importance of true model learning

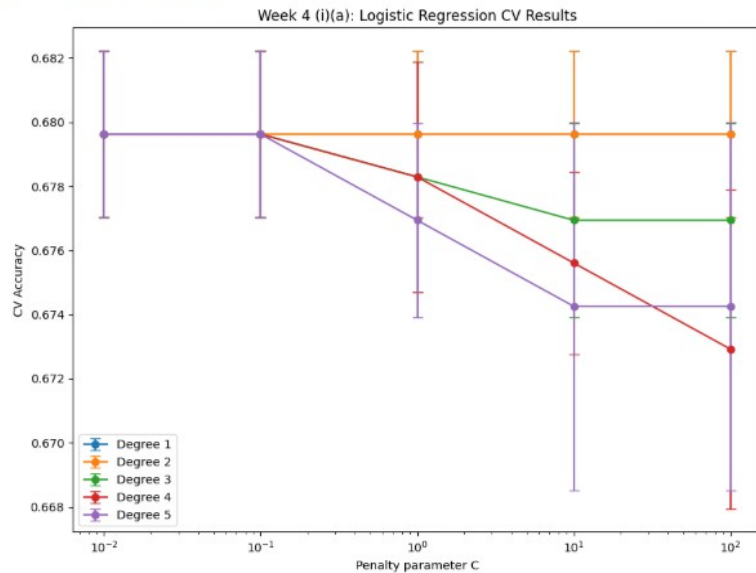
(d) I train logistic regression, Knn and two baseline models to get ROC curves by using roc curve function.



(e) After seeing the confusion matrix, Logistic Regression and kNN model are much better than baseline models, but the difference between Logistic Regression model and kNN model is very small. According to the ROC curves, the ROC curve of logistic regression model and that of Knn model is almost very similar. However the ROC curve of random baseline is a line and curve of logistic regression and kNN are actual curves. For this dataset, both of the logistic @regression and Knn model are recommended. Because according to the confusion matrix and ROC curve, these two models show good performance.

(ii) (a)

	0	1
count	746.000000	746.000000
mean	0.009142	0.004155
std	0.578277	0.568804
min	-1.000000	-1.000000
25%	-0.510000	-0.500000
50%	0.075000	0.010000
75%	0.460000	0.490000
max	1.000000	1.000000

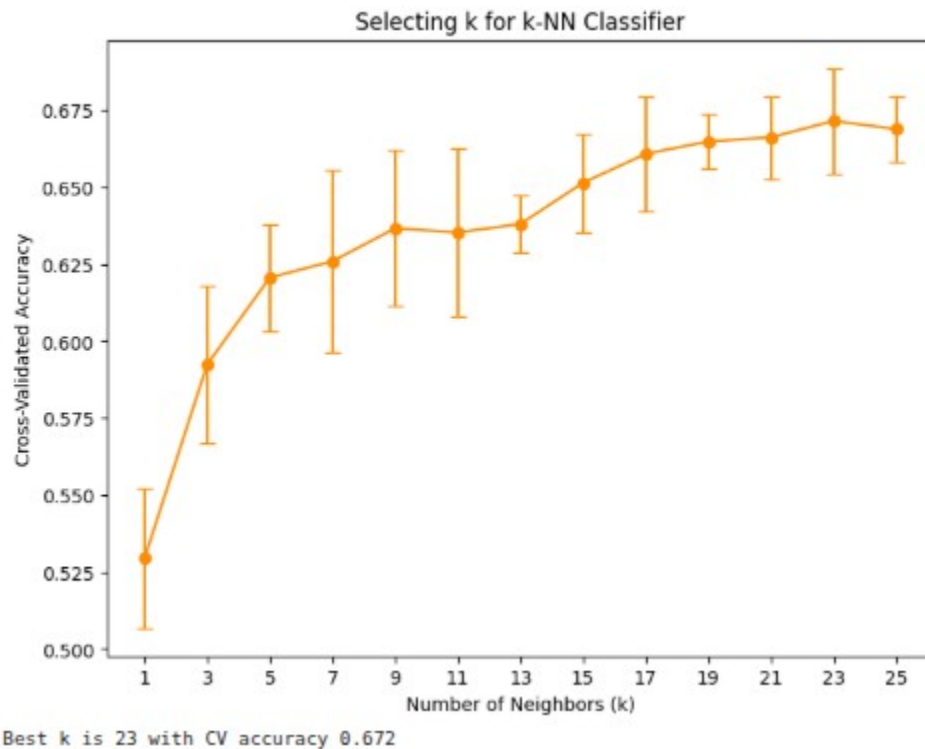


Best hyperparameters by CV accuracy:

degree	1.000000
C	0.010000
mean_acc	0.679624
std_acc	0.002606
Name: 0, dtype: float64	

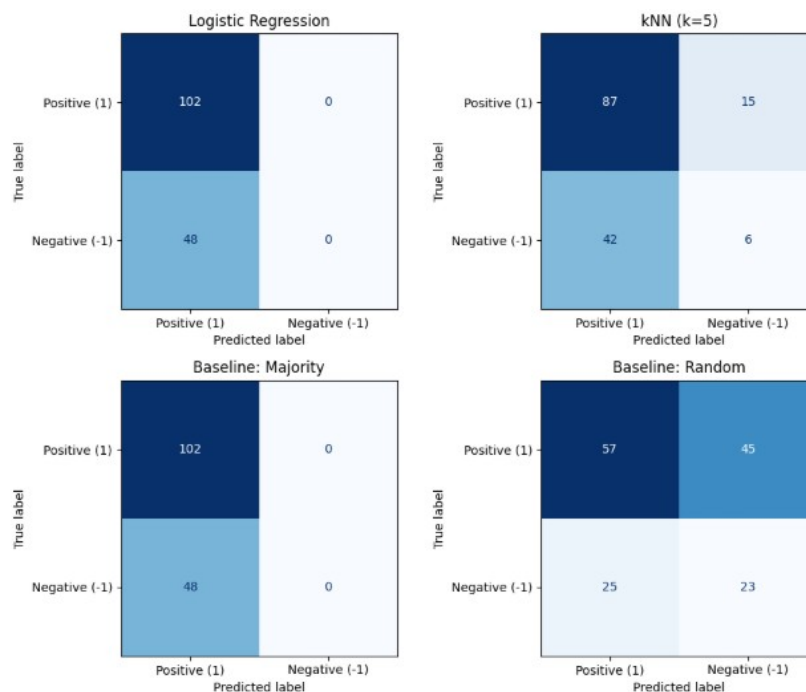
The code logic is the same just the dataset is different. The result is as shown in the image above. The best result was achieved using polynomial degree 1 and $C = 0.01$ with mean accuracy of 0.678. Increasing this polynomial degree did not improve any accuracy. This means that larger values of C did not increase performance and sometimes led to slightly higher variability.

(b)

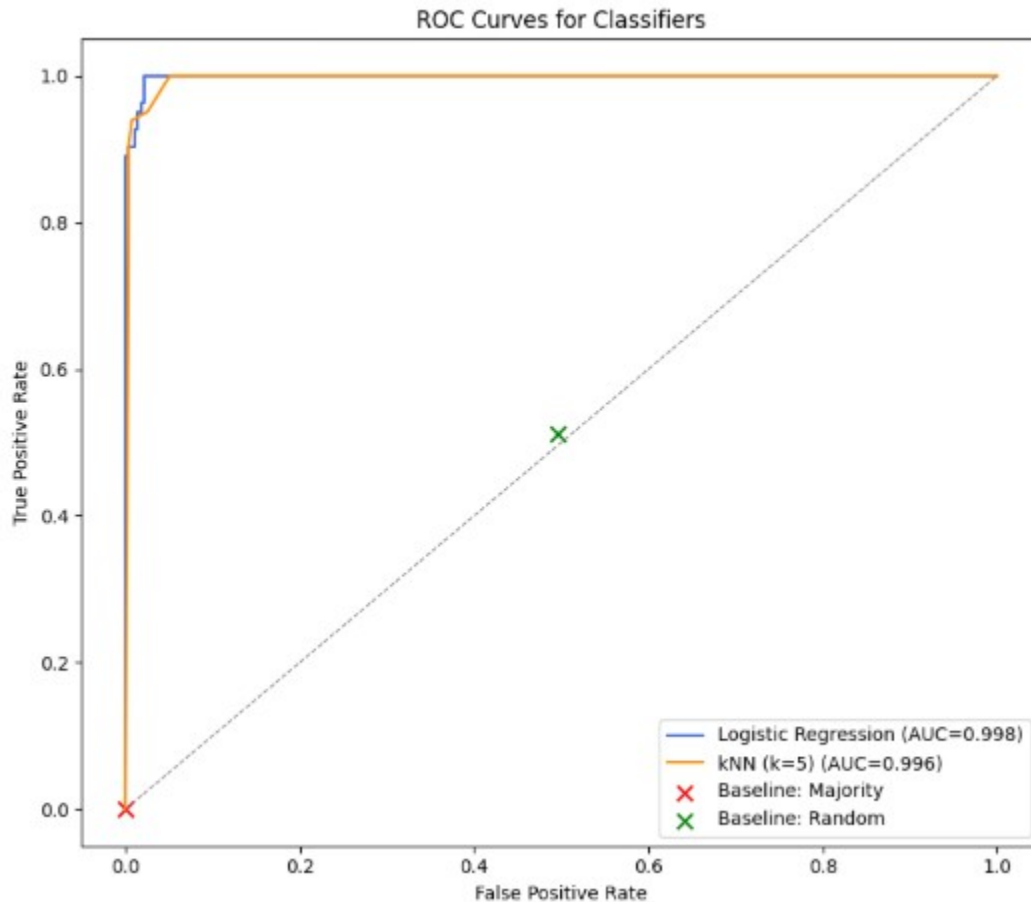


In dataset 2, the best performance was achieved with $k=23$, resulting in a mean cross validation accuracy of 0.672. Lower values of k produced lower accuracy but there is improvement in accuracy in larger values of k . This means that the class boundaries are not highly localized.

(c)



(d)



The ROC curves of the second data set can be shown In the image above.

(e) According to the confusion matrix, no classifier is significantly better or worse than others. According to the ROC curves we can see that all of these curves are not much different than others. For this dataset I recommend the kNN model because according to the performance in confusion matrix and roc model, this model shows the best performance compared to others.

Appendix

(a)

```
import pandas as pd
import numpy as np
from sklearn.preprocessing import PolynomialFeatures
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import cross_val_score
import matplotlib.pyplot as plt

df = pd.read_excel('week4_dataset1.xlsx')

X = df.iloc[:, :2].values
y = df.iloc[:, 2].values

print("First few rows:\n", df.head())
print("Class distribution:", pd.Series(y).value_counts())
print("Feature stats:\n", pd.DataFrame(X).describe())

degrees = [1, 2, 3, 4, 5]
C_values = [0.01, 0.1, 1, 10, 100]

results = []
for degree in degrees:
    poly = PolynomialFeatures(degree=degree, include_bias=False)
    X_poly = poly.fit_transform(X)
    for C in C_values:
        model = LogisticRegression(C=C, max_iter=5000)

        scores = cross_val_score(model, X_poly, y, cv=5, scoring='accuracy')
        mean_acc = np.mean(scores)
        std_acc = np.std(scores)
        results.append({'degree': degree, 'C': C, 'mean_acc': mean_acc, 'std_acc': std_acc})

res_df = pd.DataFrame(results)
plt.figure(figsize=(9,7))
for degree in degrees:
    subset = res_df[res_df['degree']==degree]
    plt.errorbar(subset['C'], subset['mean_acc'], yerr=subset['std_acc'], label=f'Degree {degree}', marker='o', capsize=4)
plt.xscale('log')
plt.xlabel('Penalty parameter C')
plt.ylabel('CV Accuracy')
```



```

plt.title('Week 4 (i)(a): Logistic Regression CV Results')
plt.legend()
plt.tight_layout()
plt.show()

best = res_df.loc[res_df['mean_acc'].idxmax()]
print("\nBest hyperparameters by CV accuracy:")
print(best)

```

(b)

```

import pandas as pd

import numpy as np
from sklearn.neighbors import KNeighborsClassifier
from sklearn.model_selection import cross_val_score
import matplotlib.pyplot as plt

df = pd.read_excel('week4_dataset1.xlsx')
X = df.iloc[:, :2].values
y = df.iloc[:, 2].values

k_values = list(range(1, 26, 2))
mean_acc = []
std_acc = []

for k in k_values:
    knn = KNeighborsClassifier(n_neighbors=k)
    scores = cross_val_score(knn, X, y, cv=5, scoring='accuracy')
    mean_acc.append(np.mean(scores))
    std_acc.append(np.std(scores))

plt.figure(figsize=(8,6))
plt.errorbar(k_values, mean_acc, yerr=std_acc, fmt='-o', color='darkorange',
             capsize=4)
plt.xlabel('Number of Neighbors (k)')
plt.ylabel('Cross-Validated Accuracy')
plt.title('Selecting k for k-NN Classifier')
plt.xticks(k_values)
plt.show()

best_k_index = np.argmax(mean_acc)
best_k = k_values[best_k_index]
best_acc = mean_acc[best_k_index]

```

```
print(f"Best k is {best_k} with CV accuracy {best_acc:.3f}")
```

(c)

```
import pandas as pd
from sklearn.preprocessing import PolynomialFeatures
from sklearn.linear_model import LogisticRegression
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import confusion_matrix, ConfusionMatrixDisplay
from sklearn.dummy import DummyClassifier
from sklearn.model_selection import train_test_split
import matplotlib.pyplot as plt

df = pd.read_excel('week4_dataset1.xlsx')
X = df.iloc[:, :2].values
y = df.iloc[:, 2].values

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, stratify=y,
random_state=42)

poly = PolynomialFeatures(degree=2, include_bias=False)
X_train_poly = poly.fit_transform(X_train)
X_test_poly = poly.transform(X_test)

log_model = LogisticRegression(C=10, max_iter=5000)
log_model.fit(X_train_poly, y_train)
y_pred_log = log_model.predict(X_test_poly)

best_k = 5
knn_model = KNeighborsClassifier(n_neighbors=best_k)
knn_model.fit(X_train, y_train)
y_pred_knn = knn_model.predict(X_test)

baseline_majority = DummyClassifier(strategy='most_frequent')
baseline_majority.fit(X_train, y_train)
y_pred_base_majority = baseline_majority.predict(X_test)

baseline_random = DummyClassifier(strategy='uniform', random_state=42)
baseline_random.fit(X_train, y_train)
y_pred_base_random = baseline_random.predict(X_test)

fig, axes = plt.subplots(2, 2, figsize=(10,8))
titles = ['Logistic Regression', f'kNN (k={best_k})', 'Baseline: Majority', 'Baseline:
Random']
```

```

preds = [y_pred_log, y_pred_knn, y_pred_base_majority, y_pred_base_random]

for ax, title, pred in zip(axes.ravel(), titles, preds):
    cm = confusion_matrix(y_test, pred, labels=[1, -1])
    disp = ConfusionMatrixDisplay(confusion_matrix=cm, display_labels=['Positive (1)',
                              'Negative (-1)'])
    disp.plot(ax=ax, cmap='Blues', colorbar=False)
    ax.set_title(title)
    plt.tight_layout()
    plt.show()

```

(d)

```

import pandas as pd
import numpy as np
from sklearn.preprocessing import PolynomialFeatures
from sklearn.linear_model import LogisticRegression
from sklearn.neighbors import KNeighborsClassifier
from sklearn.dummy import DummyClassifier
from sklearn.model_selection import train_test_split
from sklearn.metrics import roc_curve, RocCurveDisplay, auc

df = pd.read_excel('week4_dataset1.xlsx')
X = df.iloc[:, :2].values
y = df.iloc[:, 2].values
X_train, X_test, y_train, y_test = train_test_split(X, y, stratify=y, test_size=0.2,
random_state=42)

poly = PolynomialFeatures(degree=2, include_bias=False)
X_train_poly = poly.fit_transform(X_train)
X_test_poly = poly.transform(X_test)
logreg = LogisticRegression(C=10, max_iter=5000)
logreg.fit(X_train_poly, y_train)
prob_log = logreg.predict_proba(X_test_poly)[:, 1]

best_k = 5
knn = KNeighborsClassifier(n_neighbors=best_k)
knn.fit(X_train, y_train)
prob_knn = knn.predict_proba(X_test)[:, 1]

dummy_maj = DummyClassifier(strategy='most_frequent')
dummy_maj.fit(X_train, y_train)
prob_maj = dummy_maj.predict_proba(X_test)[:, 1]

dummy_rand = DummyClassifier(strategy='uniform', random_state=42)

```

```

dummy_rand.fit(X_train, y_train)
prob_rand = dummy_rand.predict_proba(X_test)[:, 1]

fpr_log, tpr_log, _ = roc_curve(y_test, prob_log, pos_label=1)
fpr_knn, tpr_knn, _ = roc_curve(y_test, prob_knn, pos_label=1)

maj_pred = dummy_maj.predict(X_test)
rand_pred = dummy_rand.predict(X_test)
fpr_maj = np.sum((maj_pred == 1) & (y_test == -1)) / np.sum(y_test == -1)
tpr_maj = np.sum((maj_pred == 1) & (y_test == 1)) / np.sum(y_test == 1)
fpr_rand = np.sum((rand_pred == 1) & (y_test == -1)) / np.sum(y_test == -1)
tpr_rand = np.sum((rand_pred == 1) & (y_test == 1)) / np.sum(y_test == 1)

plt.figure(figsize=(8,7))
plt.plot(fpr_log, tpr_log, label=f'Logistic Regression (AUC={auc(fpr_log,
tpr_log):.3f})', color='royalblue')
plt.plot(fpr_knn, tpr_knn, label=f'kNN (k={best_k}) (AUC={auc(fpr_knn,
tpr_knn):.3f})', color='darkorange')
plt.scatter([fpr_maj], [tpr_maj], c='red', label='Baseline: Majority', s=70,
marker='x')
plt.scatter([fpr_rand], [tpr_rand], c='green', label='Baseline: Random', s=70,
marker='x')
plt.plot([0,1],[0,1], '--', color='gray', lw=0.8)
plt.xlabel('False Positive Rate')
plt.ylabel('True Positive Rate')
plt.title('ROC Curves for Classifiers')
plt.legend()
plt.tight_layout()
plt.show()

```

